CoderAxo — Assignment 2: End-to-End AI Application Development

Project: DrowsyGuard — Real-Time Drowsiness Monitor

Intern: Muhammad Azeen Waqas (CAX-OL-2026-290)

1. Frontend Development

Requirement: Develop a professional frontend using Streamlit, Flask, FastAPI, or equivalent to provide a clean interface.

Implementation:

The application utilizes a highly optimized, custom **FastAPI** backend coupled with a modern **Vanilla HTML/CSS/JS** frontend.

- **WebSockets for Real-Time Streaming:** Instead of standard HTTP requests, the frontend uses WebSockets to establish a continuous, bidirectional stream with the server. This allows for zero-latency frame transmission.
- **Client-Side Rendering:** To eliminate lag, the webcam video is rendered directly on the client browser using HTML5 , while an invisible extracts and compresses frames to send to the backend for ML inference.
- **Professional UI/UX:** The interface features a clean, responsive layout with glassmorphism effects, dynamic state badges (Alert, Drowsy, Very Drowsy), and a telemetry dashboard displaying real-time metrics like Eye Aspect Ratio (EAR), Mouth Aspect Ratio (MAR), and PERCLOS.

2. Model Integration

Requirement: Connect the trained AI/ML model with the frontend for real-time predictions.

Implementation:

The **`best_model.pkl`** (trained via Scikit-Learn) is deeply integrated into the FastAPI backend (**`backend/detector.py`**).

- **Feature Extraction:** OpenCV and MediaPipe FaceMesh extract raw facial landmarks from the incoming WebSocket frames. Custom algorithms calculate EAR, MAR, and Head Pitch (using **`cv2.solvePnP`**).
- **Real-Time Inference:** These metrics are fed into the pickled Support Vector Machine (SVM) model to generate real-time probability scores.
- **Feedback Loop:** The backend immediately returns a JSON telemetry packet containing the predicted state (**`Alert`**, **`Drowsy`**, **`Very Drowsy`**), bounding box coordinates, and a "Drowsiness Score", which the frontend uses to draw visual overlays and trigger the Web Audio API alarm system.

3. Model Improvement

Requirement: Train and compare at least three different models, apply hyperparameter tuning, and select the best with justification.

Implementation:

As documented in the project's Jupyter Notebook (**`Notebook.ipynb`**), the following models were trained and compared on a 5000-sample dataset derived from YawDD characteristics:

1. **Support Vector Machine (SVM with RBF Kernel)**
2. **Random Forest Classifier**
3. **Gradient Boosting Classifier**

Hyperparameter Tuning:

A **`GridSearchCV`** was applied to the SVM model to optimize the **`C`**, **`gamma`**, and **`kernel`** hyperparameters over 5 cross-validation folds. The optimized parameters yielded the highest accuracy.

Justification for Best Model:

The **Tuned SVM** was selected as the final production model (**`best_model.pkl`**). While Random Forest offered slightly faster training times, the Tuned SVM provided a superior balance of precision and recall (crucial for minimizing false positive alarms while ensuring no actual drowsiness events were missed) and resulted in a +8.9% accuracy improvement over a standard rule-based baseline approach.

4. Performance Evaluation

Requirement: Include appropriate evaluation metrics (Accuracy, Precision, etc.).

Implementation:

The final tuned SVM model achieved exceptional performance metrics on the test split:

- **Test Accuracy:** 98.40%
- **F1-Score (Weighted):** 0.9840
- **Precision:** > 98%
- **Recall:** > 98%
- **Macro-Average AUC-ROC:** 0.9985

These metrics successfully exceed the 90% accuracy target, proving the system is highly reliable and production-ready for real-time driver monitoring.

5. Deployment & Production

Requirement: The application must be a complete, user-friendly, and production-ready AI application.

Implementation:

To demonstrate true end-to-end capabilities, the application has been deployed to the cloud:

- **Frontend (Vercel):** The HTML/JS client is hosted globally on Vercel, ensuring fast load times and SSL-secured access to the device webcam.
- **Live URL:** https://drowsiness-detection-omega.vercel.app/
- **Backend (Railway):** The heavy ML processing and FastAPI server run in a Dockerized environment on Railway, communicating seamlessly with the Vercel frontend via secure WebSockets (**`wss://`**).
- **Live API URL:** https://drowsiness-detection-production-98d6.up.railway.app/

6. Project Submission Links

- **Google Drive Project Folder (Source Code, Demo Video & Report):**
  https://drive.google.com/drive/folders/1wxBx_gn_X-MOUlmefHjdpQLtRjbdpJYk?usp=sharing